

Morse Code Translator

Harry Rudd, Daniel Craig, Ryan Shipwash, Mashoud Moussa Modi, Daniel Davis

College of Computer and Electrical Engineering, Tennessee Technological University

Hprudd42@tntech.edu

drcraig43@tntech.edu

rmshipwash42@tntech.edu

mmoussamo42@tntech.edu

ddavis44@tntech.edu

Table of Contents

I.	Introduction	2
II.	Project Specification and description	3
III.	Detailed Implementation	3
IV.	Analysis	4
V.	Description of the Teamwork Experience	4

I. INTRODUCTION

This project implements a Morse code system that uses a Nuclueo-L452RE is programmed using Keil uVision 5. It converts letters (a-z) inputted by the users to its corresponding Morse code by blinking an LED and activating a buzzer. Application of this project include secure and reliable communication system that is actively used in military, aviation, and maritime communications. Moreover, it can also be used as a morse code teaching tool, and assistive technology for speech impaired individuals.

In this project, we support the input of letters and spaces using the Nucleo keypad buttons and signal the code through both LED blinking and buzzer sound in real time. The biggest challenge we faced was converting the letters into their appropriate dot and dashes sequences and ensuring that the timing of both the buzzer and LED match the morse code protocols. We overcame this challenge by converting the letters into morse code and the morse code into binary. And using the morse binary code we the used the 11 to turn on the LED and buzzer, 00 to turn of the LED and buzzer, 01 to go the next letter.

II. PROJECT SPECIFICATION AND DESCRIPTION

The user can input characters using a keypad interface on the Nucleo-L452RF reminiscent of old mobile phone texting. As shown in Fig.1, each of buttons 2–9 corresponds to three or four letters, and quick presses of the button select the intended letter (e.g., one press of 2 produces 'A', two presses produce 'B', and three presses produce 'C'). Button 1 commits the current letter (space/commit), D is backspace, and A starts playing back Morse code. Once a message is entered on the keypad, typing 'A' allows the SysTick to read the codes stored in the morse code array. We converted morse code into a binary system where two bits represent a unit, and every unit is a single SysTick cycle. Dots are represented by the binary 11, and dashes by 111111. Pause between a character is 00 and pause between letters is 01. The system turns on the LED and buzzer upon reading 11 in a Morse character, turns them off for 00 according to these binary strings, and 01 at the end of each letters tells the system to go to the next letter.

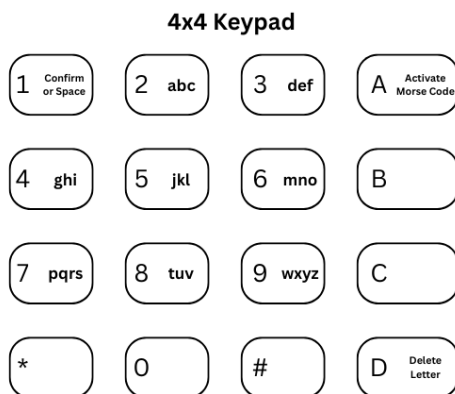


Fig. 1. Keypad Layout

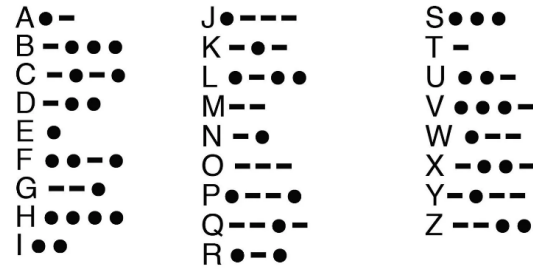


Fig. 2. Morse Code Chart

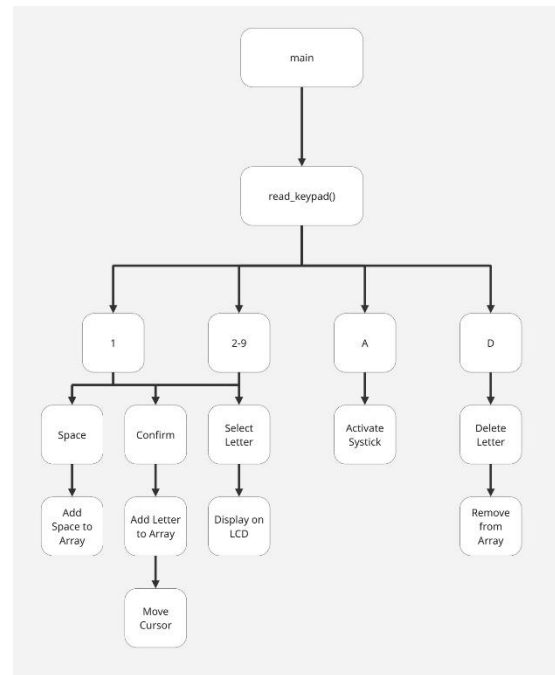


Fig. 3. Main Function Block Diagram

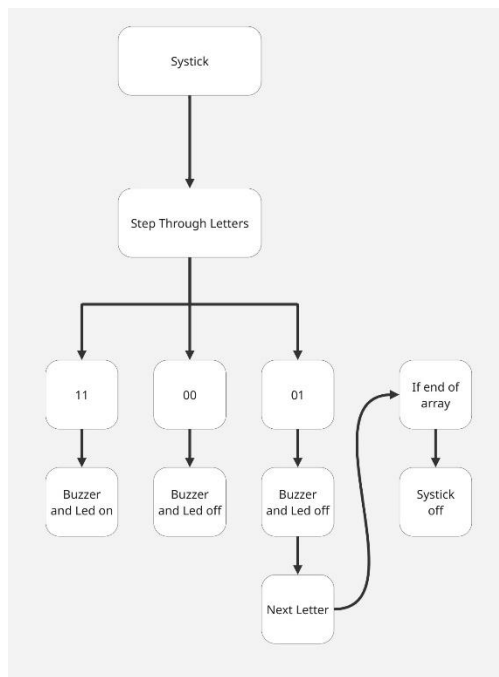


Fig. 4. Systick Function Block Diagram

III. DETAILED IMPLEMENTATION

A. Main:

- Create a variable called “wait” the acts as the activation for the Systick and initialize it to 1.
- Create a variable called “step” that acts as a shift for the pairs of bits while reading through the morse code letters.
- Create an unsigned 32-bit integer array with 16 values. This array is going to hold the hexadecimal morse code values for the letters or spaces selected and confirmed.
- Create a morse index for the current array position and initialize it to 0.
- Create a comparison index for stepping through the array while outputting the morse code.
- Read the keypad input.
- Input cases:
 - If ‘1’ is pressed without a selected letter, the hexadecimal morse value for ‘space’ is added to the array, and a space is added to the display (moving the cursor forward). If one is pressed while a letter is selected, the letter will be confirmed and added to the array also moving the cursor forward. In both instances, an index is incremented by 1 to the next location in the array.
 - If ‘2-9’ is pressed, a letter is selected and displayed on the display. By pressing one of these numbers multiple times without confirming, it will cycle through the letters assigned to that number like in a phone

keypad showing the cycling on the display by incrementing a cycle value (the displayed letter resets back to the original letter when pressed a fourth time). Once the letter is confirmed (by 1), the function “Find_Letter” is called to determine the letters associated hexadecimal value, and that value is stored in the array at the current position.

- If ‘D’ is pressed, delete letter in the display and remove the letter from the array. The index is decremented by one to the previous location in the array.
- If A is pressed, the “wait” variable is set to 0, activating functions in the Systick.

B. Find_Letter:

- Takes in the number on the keypad that was pressed and the cycle value to return the hexadecimal value associated with the morse code of the letter.
- We determined the morse code hexadecimal value by creating a unit system where every unit of the morse code is represented by two bits. 11 is the high value and 00 is the low value. 01 is a low value and the value that determines when the letter is finished reading. So, a dot is represented by a 1100 (00 for a 1 unit pause after), a dash is represented by 11111100 (00 for a 1 unit pause after), and every letter has 0001 (two pauses and the end of letter indicator) followed by 0’s until 32 bits have been reached for filler. We used 32 bit unsigned integers because 32 bits was the maximum needed for this system. Finally, we converted the 32 bit integers into hexadecimal to make it usable for the code.

C. Systick:

- The functions in the SysTick are called when wait is low.
- Sets the comparison index equal to the morse index to start.
- While the comparison index is greater than 0, do a bitwise and between a pair of bits in the current letter in the array (determined by the morse index subtracted by the comparison index) shifted by the step value from the most significant pair (reading left to right) to determine if each pair of bits is 11, 00, or 01. The first if statement compares 11, the second compares 01, and the else determines that the pair is 00.
- If a pair is 11, the buzzer and external LED are set to high, and the step is incremented.
- If a pair is 00, the buzzer and external LED are set to low, and the step is incremented.

- If a pair is 01, the buzzer and external LED are set to low, step is set to 1, comparison index is decremented to step to the next letter. Finally, once the comparison index is 0 again, set wait to 1 to deactivate the SysTick, clear the display, and set the number pressed value to 1.

D. Buzzer:

- We initialized a timer interrupt to activate rapidly as the program is running.
- The priority of the timer is set to be lower than that of the systick, so the systick can consistently change the buzzer flag.
- If the buzzer flag is running, then toggle the buzzer pin. This rapid oscillation creates a tone that is played when the buzzer is allowed to run.

E. LED:

- External LED is connected from the cathode to PB0, and the anode is connected to a 1 kilohm resistor that is connected to ground. Therefore, we initialized the LED as PB0

row as expected. We also ensured that buzzer toggle in sync with the LED. This helped verify that both visual and audio outputs were synchronized and produced the intended behavior.

The design of this project considers public safety and education. It can be used to teach students morse code for military or just educational purposes. The translator is made to be simple and cost-effective. It also is a communication alternative for people with vision or speech impairments. The buzzer provides auditory feedback while the LED provides visual feedback. It also features minimal power consumption and minimal user interaction.

The design of this project considers global factors because it helps with international communication. This design is also cost effective and requires minimal power, so it is suitable for underserved and remote areas.

To further improve our project, we could add potentiometer so the user could control the speed of the systick, which would increase or decrease the speed at which the morse code is displayed. This would allow the user to control the speed of the morse code- slower for beginners or faster for experienced users.

IV. ANALYSIS

To test the input and the functionality of the keypad and matrix, we added and removed characters at random which ensured that each keypress was correctly detected and registered. To test our design, we did a stress test after loading the project to the board. We performed this test with the letter "O", because the morse code of the letter "O" is three long dashes as shown in Fig. 2. This allowed us to confirm that the LED stayed on for an extended period of time three times in a

V. DESCRIPTION OF THE TEAMWORK EXPERIENCE

Every team member contributed to a good and productive working atmosphere by speaking openly and working to each other's strong points. We used a group chat to coordinate meetings, and we helped each other in areas where we were slacking. We took turns leading different parts of the project, set goals clearly along the way, and stayed organized with regular check-ins to make sure that everything was still on track.